Day 1 — Hands-on Problem Statements

Session 1: Fundamentals

Practical 1 — Syntax, Variables & Datatypes

Write a program to accept a student's roll number, name, age, and CGPA (using appropriate datatypes) and display them in a formatted manner.

Practical 2 — Operators (including Bitwise)

Write a program to accept two integers and:

- Perform all arithmetic operations (+, -, *, /, %)
- Perform bitwise operations (AND, OR, XOR, NOT, left shift, right shift) on them
- Check whether a given number is even or odd using the bitwise AND operator (instead of modulus)

Practical 3 — Conditional Statements (if, else, Nested if, switch)

- Write a program to check whether a given year is a leap year (nested if)
- Write a menu-driven program using switch-case to perform basic arithmetic operations (+, -, *, /) based on user's choice

Practical 4 — Loop Statements

- Write a program to print the Fibonacci series up to N terms
- Write a program to check whether a given number is prime
- Write a program to print the following pattern using nested loops:
- 11 21 2 31 2 3 4

Session 2: Functions, Arrays & Pointers

Practical 5 — User Defined Function

Write a function int factorial(int n) to calculate the factorial of a number, and call it from main() to display results for numbers 1 to 10.

Practical 6 — Arrays (int, character, two-dimensional)

- Write a program to find the largest and smallest element in an integer array
- Write a program to check whether a given string (character array) is a palindrome
- Write a program to add two 3x3 matrices using two-dimensional arrays

Practical 7 — Pointers (call by value vs call by reference)

- Write a program demonstrating call by value vs call by reference by writing two versions of a swap() function
- Write a program to access and print array elements using pointer arithmetic instead of array indexing

Session 3: Structures & Strings

Practical 8 — Structures
Write a program using a struct student (with roll number, name, and marks) to:

- Accept and display details of 3 students
- Find and display the student with the highest marks

Practical 9 — Strings

- Write a program to count the number of vowels, consonants, digits, and spaces in a given string
- Write a program to reverse a string without using a built-in reverse function
- Write a program to check if two strings are anagrams of each other

Session 4: Introduction to OOPs

Practical 10 — Class & Objects, Data Hiding, Constructors
Design a class BankAccount with private data members (account number, holder name, balance):

- Implement accessor and mutator functions (data hiding)
- Implement a parameterized constructor to initialize account details
- Implement deposit() and withdraw() member functions with balance validation

Practical 11 — Pointer to Object, All Types of Functions in Class
Using the BankAccount class above:

- Create an object dynamically using a pointer (new) and access members using the arrow operator
- Add a static-like utility function inside the class to display the bank's name (demonstrating a function that doesn't depend on object state)

Practical 12 — Scope Resolution Operator & Inline Function

- Define a class Rectangle with member function declarations inside the class and definitions outside using the scope resolution operator (::)
- Write an inline function getArea() inside the same class and observe/discuss the use of the this pointer when a parameter name shadows a member name

Practical 13 — Struct vs Class
Rewrite the student structure (Practical 8) as a class with private members and public accessor/mutator functions. Compare and note the differences in default access specifiers and design intent between struct and class.

Day 2 — Hands-on Problem Statements

Session 1: Operator Overloading

Practical 1 — Operator Overloading (Binary Operator)
Design a class Complex (with real and imaginary parts) and overload the + operator to add two complex numbers. Overload the == operator to check if two complex numbers are equal.

Practical 2 — Friends (as helper for operator overloading)
Using the Complex class above, overload the + operator as a **friend function** instead of a member function, and explain/discuss why a friend function is needed when the left operand isn't an object of the class (e.g., 2 + c1).

Practical 3 — Insertion Operator Overloading
Overload the << (insertion) operator for the Complex class so that cout << c1; directly prints the complex number in the form a + bi, instead of writing a separate display function.

Session 2: Inheritance

Practical 4 — Basic Inheritance & Access Specifiers
Create a base class Person (name, age) and derive a class Employee (salary, department) from it using **public inheritance**. Demonstrate how private, protected, and public members of Person are accessed in Employee and in main().

Practical 5 — Constructor in Inheritance
Extend the Person–Employee example to include constructors in both classes. Demonstrate the order of constructor calls (base before derived) by printing a message inside each constructor, and pass values from the derived class constructor to the base class constructor.

Practical 6 — IS-A vs HAS-A
- Implement an **IS-A** relationship: Manager IS-A Employee (inheritance)
- Implement a **HAS-A** relationship: Employee HAS-A Address (composition — an Address object as a member of Employee)
- Discuss when to prefer inheritance over composition

Practical 7 — Ways of Inheritance (public, private, protected)
Write three versions of a class Base derived as public, private, and protected in three separate derived classes. Try accessing base class members from main() in each case and observe/document the compilation differences.

Practical 8 — Generalization & Specialization
Design a class hierarchy: Shape (generalized base class) → Circle, Rectangle, Triangle (specialized derived classes), each implementing their own area() calculation. (This sets up the transition into polymorphism.)

Session 3: Polymorphism

Practical 9 — Base Class Pointer, Derived Class Object

Using the Shape hierarchy from Practical 8, create a Shape* pointer and assign it objects of Circle, Rectangle, and Triangle one at a time. Call area() through the base pointer and observe the default (non-virtual) behavior.

Practical 10 — Function Overriding & Virtual Functions
Modify the Shape class to declare area() as a **virtual function**. Override it in Circle, Rectangle, and Triangle. Demonstrate the difference in output compared to Practical 9.

Practical 11 — Runtime Polymorphism
Create an array of Shape* pointers pointing to different derived class objects (Circle, Rectangle, Triangle). Loop through the array and call area() on each, demonstrating that the correct overridden function is called at runtime based on the actual object type.

Practical 12 — Abstract Classes
Convert Shape into an **abstract class** by making area() a pure virtual function (virtual double area() = 0;). Attempt to instantiate Shape directly (observe the compiler error) and confirm that only derived classes implementing area() can be instantiated.

Session 4: Friends, Static & Exception Handling

Practical 13 — Friend Function & Friend Class

- Write a friend function that accesses private members of two different classes (e.g., a function to compare private "distance" values from two Box class objects)
- Declare one class as a **friend class** of another and demonstrate direct access to private members

Practical 14 — Static Members
Design a class Counter with a static data member count that increments every time an object is created (in the constructor). Display the total number of objects created so far using a static member function.

Practical 15 — Nested Class
Design an Engine class nested inside a Car class. Create an object of Car that internally creates and uses an Engine object, demonstrating how nested classes model tightly coupled "part-of" relationships.

Practical 16 — Exception Handling (throw and catch, between functions)
Write a function divide(int a, int b) that throws an exception (throw) when b == 0. Call this function from another function, which in turn is called from main(), and catch the exception in main() — demonstrating exception propagation **between functions** (not just within the same function).

Day 3 — Hands-on Problem Statements

Session 1: Language Utilities

Practical 1 — Template Functions & Classes

- Write a **template function** getMax(T a, T b) that returns the larger of two values, and test it with int, float, and char arguments
- Write a **template class** Pair<T> that stores two values of the same type and provides a function to display them

Practical 2 — Constant Qualifier

Write a program demonstrating:

- A const variable that cannot be modified after initialization
- A const member function in a class that cannot modify the object's data members
- A function accepting a const reference parameter to prevent accidental modification

Practical 3 — Preprocessor Directives

- Write a program using #define to create a macro for calculating the square of a number
- Use conditional compilation (#ifdef, #ifndef, #endif) to include/exclude a debug print statement

Practical 4 — Namespace

Create two namespaces, Math and Physics, each containing a function/constant with the same name (e.g., PI). Demonstrate accessing both using the scope resolution operator, and using a using namespace directive.

Practical 5 — Destructor & Virtual Destructor

- Write a class with a constructor and destructor that print messages on object creation/destruction
- Demonstrate the problem of **not** having a virtual destructor: create a base class pointer pointing to a derived class object, delete it, and observe that the derived destructor is skipped
- Fix it by making the base class destructor virtual and observe the corrected output

Session 2: File Handling

Practical 6 — File Stream (Text Files — Writing & Reading)

Write a program to accept details of 3 students (name, marks) from the user and write them to a text file students.txt. Then read and display the contents of the file.

Practical 7 — File Stream (Binary Files)

Write a program to store Employee objects (using a struct/class) into a **binary file** using write(), and then read them back using read(), displaying all employee records.

Practical 8 — Manipulators

Write a program that uses stream manipulators (setw, setprecision, fixed, endl) to display a formatted table of student names and marks with proper column alignment and fixed decimal precision.

Session 3: STL

Practical 9 — Data Structures & Types (Vector, List)

- Write a program using vector<int> to store, sort, and display a list of numbers
- Write a program using list<int> to insert and delete elements at both ends

Practical 10 — STL Classes (Stack, Queue)

- Write a program to reverse a string using the STL stack
- Write a program to simulate a ticket counter queue using the STL queue

Practical 11 — Map Classes

Write a program using map<string, int> to store student names as keys and their marks as values. Demonstrate insertion, searching for a student by name, updating marks, and iterating over all entries.

Session 4: C++11 & Beyond

Practical 12 — Auto & Final Keyword

- Rewrite the loop from Practical 9 (vector iteration) using auto instead of explicit iterator types
- Create a class marked final and attempt to derive from it (observe the compiler error); mark a virtual function as final in a base class and attempt to override it in a derived class

Practical 13 — Lambda Expression

- Write a lambda expression to add two numbers and store it in an auto variable
- Use a lambda with the STL sort() function to sort a vector of numbers in **descending** order

Practical 14 — Smart Pointers

Write a program demonstrating unique_ptr and shared_ptr to manage a dynamically allocated object (e.g., a Student object), showing automatic memory deallocation without an explicit delete.

Practical 15 — In-Class Initializer & Delegating Constructor

Design a class Config with in-class member initializers (e.g., int timeout = 30;). Add a delegating constructor that calls another constructor of the same class to avoid duplicate initialization code.

Practical 16 — Ellipsis (va_list, va_start, va_arg, va_end)

Write a variadic function int sum(int count, ...) that accepts a variable number of integer arguments and returns their sum, using va_list, va_start, va_arg, and va_end.

Session 5: Miscellaneous (Buffer/Wrap-up)

Practical 17 — Number Systems & Conversions

Write a menu-driven program to convert a given number between Decimal, Binary, Octal, and Hexadecimal (user selects source and target format).

Practical 18 — Data Types, Word Size & Storage
Write a program using the sizeof operator to display the size (in bytes) of int, short, long, long long, float, and double on the current system, and print the minimum/maximum range of each using climits/cfloat (or <limits>).

Practical 19 — Variables & Literals
Write a program demonstrating different literal types — integer literals (decimal, octal 0, hex 0x), character literals, floating-point literals, and string literals — and print each with its type-appropriate format specifier/stream.

Practical 20 — Constructor in Inheritance (Consolidation)
Design a base class Vehicle and derived class Car, each with both a **default constructor** and a **parameterized constructor**.
Demonstrate all four combinations of constructor calls (default→default, parameterized→parameterized, etc.) by explicitly invoking the appropriate base constructor from the derived class's initializer list.